

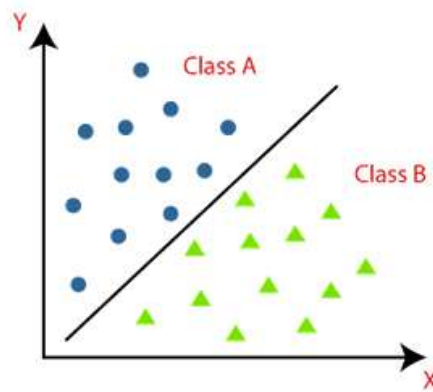
What is the Classification Algorithm?

The Classification algorithm is a Supervised Learning technique that is used to identify the category of new observations on the basis of training data. In Classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups. Such as, Yes or No, 0 or 1, Spam or Not Spam, cat or dog, etc. Classes can be called as targets/labels or categories.

$$y = f(x), \text{ where } y = \text{categorical output}$$

The main goal of the Classification algorithm is to identify the category of a given dataset, and these algorithms are mainly used to predict the output for the categorical data.

Classification algorithms can be better understood using the below diagram. In the below diagram, there are two classes, class A and Class B. These classes have features that are similar to each other and dissimilar to other classes.



There are two types of Classifications:

Binary Classifier: If the classification problem has only two possible outcomes, then it is called as Binary Classifier.

Examples: YES or NO, MALE or FEMALE, SPAM or NOT SPAM, CAT or DOG, etc.

Multi-class Classifier: If a classification problem has more than two outcomes, then it is called as Multi-class Classifier.

Learners in Classification Problems:

In the classification problems, there are two types of learners:

Lazy Learners: Lazy Learner firstly stores the training dataset and wait until it receives the test dataset. In Lazy learner case, classification is done on the basis of the most related data stored in the training dataset. It takes less time in training but more time for predictions.

Example: K-NN algorithm, Case-based reasoning

Eager Learners: Eager Learners develop a classification model based on a training dataset before receiving a test dataset. Opposite to Lazy learners, Eager Learner takes more time in learning, and less time in prediction.

Example: Decision Trees, Naïve Bayes, ANN.

Types of ML Classification Algorithms:

Classification Algorithms can be further divided into the Mainly two category:

Linear Models

1. Logistic Regression
2. Support Vector Machines

Non-linear Models

1. K-Nearest Neighbours
2. Kernel SVM
3. Naïve Bayes
4. Decision Tree Classification
5. Random Forest Classification

Support Vector Machines (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

Key Concepts of Support Vector Machine

Hyperplane: A decision boundary separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.

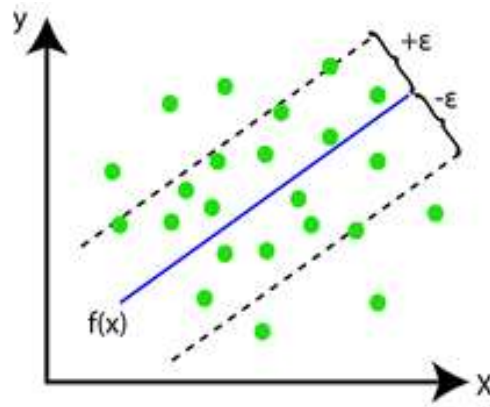
Support Vectors: The closest data points to the hyperplane, crucial for determining the hyperplane and margin in SVM.

Margin: The distance between the hyperplane and the support vectors. SVM aims to maximize this margin for better classification performance.

Kernel: A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.

Hard Margin: A maximum-margin hyperplane that perfectly separates the data without misclassifications.

Soft Margin: Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.



Here, the blue line is called hyperplane, and the other two lines are known as boundary lines or margin.

The SVM working can be explained on the basis of the below algorithm:

Step 1: Draw all the points on one plane

Step 2: Find the support Vector

Step 3: Add the bias value in vectors to convert 2D to 3D

Step 4: Consider parameters for every support vector like α_1 and α_2 etc.

Step 5: Find the linear equations and solve them to find the parameters value.

Step 6: For equation for hyperplane is.

Example: Suppose you are working on a project to classify houses as either **Luxury (+)** or **Standard (-)** based on two features:

1. House Size (in hundreds of sq. meters)
2. Number of Facilities (garage, garden, pool, etc.)

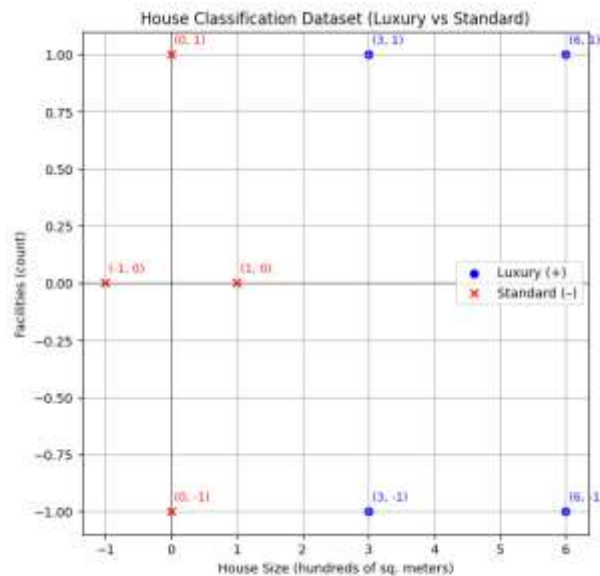
You are given the following labeled dataset:

Data Point	(House Size, Facilities)	Meaning	Class Label
(3, 1)	300 sq. m, 1 facility	Luxury	+
(3, -1)	300 sq. m, 0 facilities	Luxury	+
(6, 1)	600 sq. m, 1 facility	Luxury	+
(6, -1)	600 sq. m, 0 facilities	Luxury	+
(1, 0)	100 sq. m, 0 facilities	Standard	-
(0, 1)	Very small house, 1 facility	Standard	-
(0, -1)	Very small house, 0 facilities	Standard	-
(-1, 0)	Extremely small/poor condition	Standard	-

- Using the Support Vector Machine (SVM) concept, find the optimal decision boundary (hyperplane) that separates Luxury houses (+) from Standard houses (-).
- Predict the class label for a new house with features (4, 0) (i.e., 400 sq. m, 0 facilities).

Solution:

Step 1: Place this datapoints into a graph,



Step 2: Find the Support Vectors, Support vectors are the closest points to the separating line. From geometry, the line is at $x = 2$. Closest Luxury points: (3,1), (3,-1), Closest Standard point: (1,0) Support vectors = (3,1), (3,-1), (1,0)

Step 3: Add bias value (convert into 3D vector form)

In SVM, every input point (x_1, x_2) is represented as $(x_1, x_2, 1)$, where the extra 1 is for the bias term. So support vectors become: $s_1 = (3, 1) \rightarrow \tilde{s}_1 = (3, 1, 1)$, $s_2 = (3, -1) \rightarrow \tilde{s}_2 = (3, -1, 1)$, $s_3 = (1, 0) \rightarrow \tilde{s}_3 = (1, 0, 1)$

Step 4: Assign α values

$$\begin{aligned}
 \alpha_1 \tilde{s}_1 \cdot \tilde{s}_1 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_1 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_1 &= -1 & \alpha_1(1+0+1) + \alpha_2(3+0+1) + \alpha_3(3+0+1) &= -1 \\
 \alpha_1 \tilde{s}_1 \cdot \tilde{s}_2 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_2 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_2 &= +1 & \alpha_1(3+0+1) + \alpha_2(9+1+1) + \alpha_3(9-1+1) &= 1 \\
 \alpha_1 \tilde{s}_1 \cdot \tilde{s}_3 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_3 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_3 &= +1 & \alpha_1(3+0+1) + \alpha_2(9-1+1) + \alpha_3(9+1+1) &= 1
 \end{aligned}$$

$$\begin{aligned}
 \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} &= -1 & 2\alpha_1 + 4\alpha_2 + 4\alpha_3 &= -1 \\
 \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} &= 1 & 4\alpha_1 + 11\alpha_2 + 9\alpha_3 &= 1 \\
 \alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} &= 1 & 4\alpha_1 + 9\alpha_2 + 11\alpha_3 &= 1
 \end{aligned}$$

$$\begin{aligned}
 \alpha_1 &= -3.5 \\
 \alpha_2 &= 0.75 \\
 \alpha_3 &= 0.75
 \end{aligned}$$

Step 5:

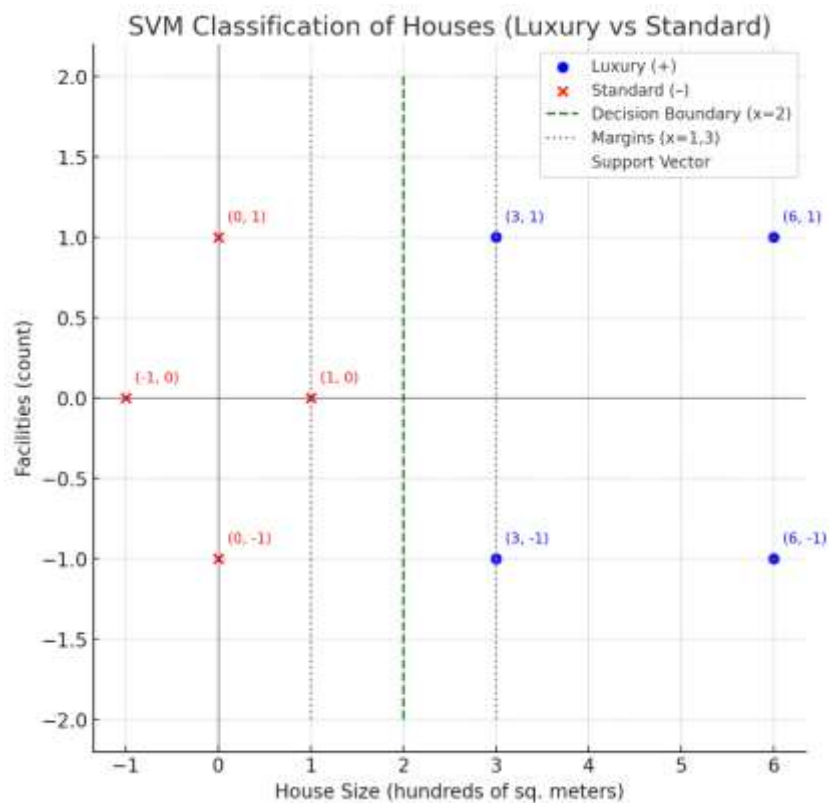
$$\begin{aligned}\tilde{w} &= \sum_i \alpha_i \tilde{s}_i \\ &= -3.5 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix}\end{aligned}$$

- Finally, remembering that our vectors are augmented with a bias.
- We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating
- Hyperplane equation $y = wx + b$
- with $w = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $b = -2$.

Step 6:

Decision boundary (hyperplane): $x - 2 = 0$, $x = 2$.

Support vectors: $(3,1)$, $(3,-1)$, $(1,0)$.



K-Nearest Neighbor (KNN) Algorithm for Machine Learning

K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.

1. K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
2. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
3. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems. K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.
4. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.

The K-NN working can be explained on the basis of the below algorithm:

Step-1: Select the number K of the neighbors

Step-2: Calculate the Euclidean distance of K number of neighbors

Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.

Step-4: Among these k neighbors, count the number of the data points in each category.

Step-5: Assign the new data points to that category for which the number of the neighbor is maximum.

Step-6: Our model is ready.

Example: We want to predict whether a movie is an **Action** or **Romance** based on two features:

1. Fight Scenes (count)
2. Love Scenes (count)

ID	Fight Scenes	Love Scenes	Genre
M1	8	2	Action
M2	7	3	Action
M3	1	9	Romance
M4	2	8	Romance
M5	6	4	Action

Solution:

Step 1: Compute Euclidean distances

Formula:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- To M1: $\sqrt{(3 - 8)^2 + (7 - 2)^2} = \sqrt{25 + 25} = 7.07$
- To M2: $\sqrt{(3 - 7)^2 + (7 - 3)^2} = \sqrt{16 + 16} = 5.66$
- To M3: $\sqrt{(3 - 1)^2 + (7 - 9)^2} = \sqrt{4 + 4} = 2.83$
- To M4: $\sqrt{(3 - 2)^2 + (7 - 8)^2} = \sqrt{1 + 1} = 1.41$
- To M5: $\sqrt{(3 - 6)^2 + (7 - 4)^2} = \sqrt{9 + 9} = 4.24$

Step 2: Sort nearest neighbors

1. M4 (1.41, Romance)
2. M3 (2.83, Romance)
3. M5 (4.24, Action)
4. M2 (5.66, Action)
5. M1 (7.07, Action)

Step 3: Apply KNN (k = 3)

Nearest 3 = M4 (Romance), M3 (Romance), M5 (Action)

Majority = 2 Romance vs 1 Action

Step 4: The new movie with 3 fight scenes & 7 love scenes is predicted to be: Romance

Step 5: Plot the graph

